

ENTERPRISE READY • PRODUCTION GRADE

Enterprise .NET Starter Kit

Manual Técnico de Arquitectura, Desarrollo y Despliegue en Producción

.NET 10 (C# 13)

Clean Architecture

CQRS & MediatR 14

EF Core 10 & PostgreSQL 17

JWT & Refresh Tokens

Polly v8 Resilience

Scalar API Reference

Docker & Compose

OWASP Hardened

Guía técnica de ingeniería de software para equipos de desarrollo, tech leads y arquitectos de soluciones. Incluye especificación de capas, ciclo de vida de autenticación criptográfica, pipeline de validación y auditoría, guía de creación de nuevos módulos y directrices DevOps.

Versión: 1.0.0 Enterprise Edition

Framework: .NET 10 LTS Target

Estado: Certificado para Producción

Calidad: 29/29 Pruebas Superadas (0 Vulnerabilidades)



Tabla de Contenidos

1. Resumen Ejecutivo & Visión General	Pág. 2
2. Arquitectura del Sistema (Clean Architecture & CQRS)	Pág. 3
3. Seguridad, Autenticación & Identity	Pág. 5
4. Persistencia, Base de Datos & Resiliencia	Pág. 7
5. Guía Paso a Paso: Cómo Agregar un Nuevo Módulo / Feature	Pág. 9
6. Estrategia de Pruebas & Calidad de Código	Pág. 12
7. Despliegue, Contenedores & Operaciones (DevOps)	Pág. 14
8. Herramientas de Productividad (DX) & Referencia de API	Pág. 16

1. Resumen Ejecutivo & Visión General

El **Enterprise .NET Starter Kit** es una base de código de ingeniería avanzada diseñada para construir aplicaciones backend escalables, mantenibles y altamente seguras bajo **.NET 10** y **C# 13**.

Certificación de Calidad y Seguridad

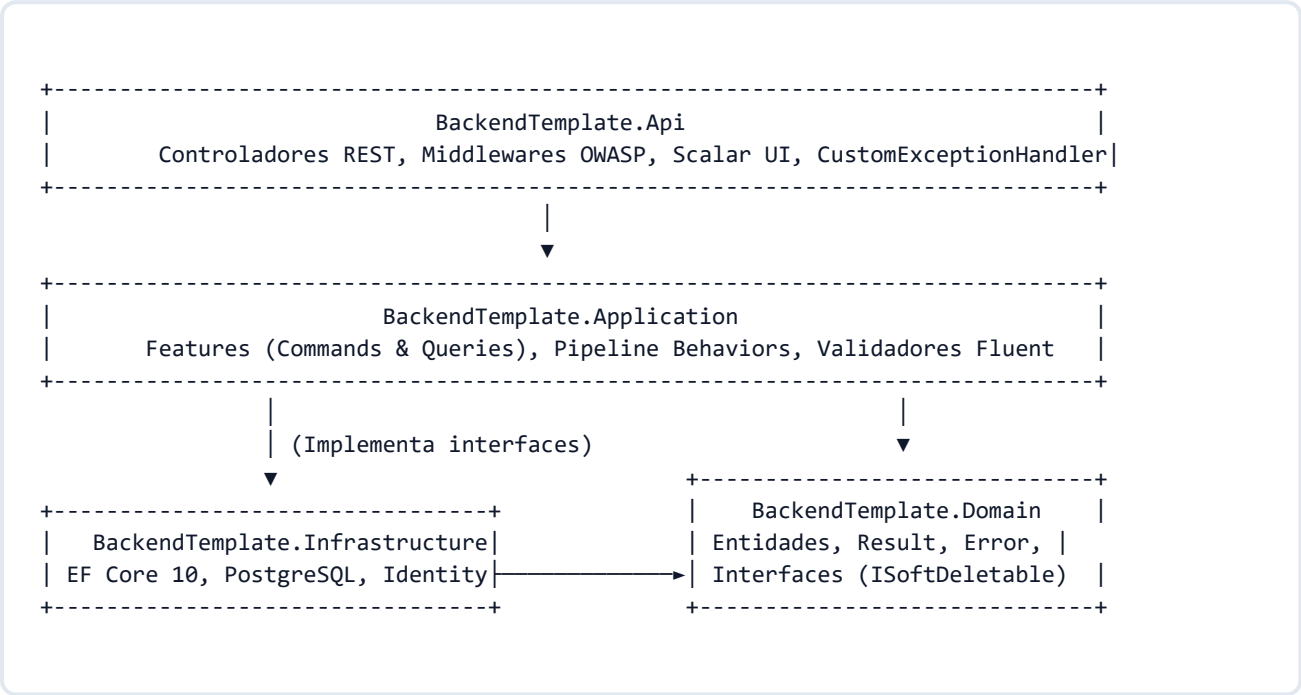
La solución ha sido auditada exhaustivamente: cuenta con **0 vulnerabilidades de paquetes (CVEs)**, **0 advertencias de compilación** y una suite completa de **29 pruebas automatizadas (100% superadas)** que verifican desde las reglas estructurales de Clean Architecture hasta endpoints HTTP de integración.

Principios Rectores del Starter Kit

- **Aislamiento de Negocio:** El dominio es puro y agnóstico a bases de datos, APIs y frameworks externos.
- **Zero Dummy Code:** Todos los ejemplos (CRUD de Estudiantes, Autenticación, Auditoría) son implementaciones reales y listas para producción.
- **Resiliencia por Diseño:** Tolerancia automática a caídas transitorias de base de datos con Polly v8.
- **Desarrollo Guiado por Estándares:** RFC 7807 para errores, OpenAPI 3.1 con Scalar para documentación, OWASP Top 10 para seguridad perimetral.

2. Arquitectura del Sistema (Clean Architecture & CQRS)

La solución está organizada en 4 capas concéntricas con dependencia unidireccional de afuera hacia adentro:



2.1. Responsabilidades por Capa

Proyecto	Responsabilidad	Dependencias Permitidas
BackendTemplate.Domain	Núcleo de negocio, entidades, objetos de valor, tipos de error (<code>Error`</code> , <code>ErrorType`</code>) y contenedor <code>Result<T>`</code> .	Ninguna (0 dependencias)
BackendTemplate.Application	Casos de uso (CQRS), contratos de interfaces (<code>IApplicationDbContext`</code> , <code>IIdentityService`</code>), validadores <code>FluentValidation</code> y pipeline behaviors.	<code>Domain`</code>
BackendTemplate.Infrastructure	Acceso a datos con EF Core, PostgreSQL (<code>Npgsql`</code>), ASP.NET Core Identity, JWT, interceptores de auditoría y migraciones.	<code>Domain`</code> , <code>Application`</code>

Proyecto	Responsabilidad	Dependencias Permitidas
BackendTemplate.Api	Capa de presentación HTTP, controladores REST, endpoints, rate limiting, middlewares de seguridad y OpenAPI / Scalar.	`Application`, `Infrastructure`

2.2. Pipeline Behaviors de MediatR

Cada comando o consulta despachada a través de `Mediator.Send(...)` atraviesa automáticamente 4 interceptores antes y después del Handler:

1. **`UnhandledExceptionBehavior`**: Captura cualquier excepción no manejada y la registra de forma estructurada con Serilog.
2. **`LoggingBehavior`**: Registra el inicio, identidad del usuario autenticado (`UserId`) y finalización de la operación.
3. **`PerformanceBehavior`**: Monitorea tiempos de respuesta y emite una alerta estructurada (`LogWarning`) si el handler tarda más de **500 ms**.
4. **`ValidationBehavior`**: Ejecuta automáticamente todos los validadores FluentValidation registrados. Si hay errores, detiene el flujo y lanza `ValidationException` antes de consultar la base de datos.

2.3. Patrón Result & Error Fuertemente Tipado

Para eliminar el costo de rendimiento de las excepciones en flujos normales de negocio, se utiliza el patrón `Result<T>` con tipos semánticos `ErrorType`:

```
// Ejemplo de retorno en Handlers
if (student == null)
{
    return Result.Failure(Error.NotFound("Students.NotFound", $"Student with Id '{id}' was not f
});

student.Delete();
await _context.SaveChangesAsync(cancellationToken);
return Result.Success();
```

3. Seguridad, Autenticación & Identity

3.1. Flujo Criptográfico de Refresh Tokens

La autenticación utiliza **JWT (HMAC-SHA256)** con rotación de Refresh Tokens de alta seguridad almacenados en la base de datos:

 Detección de Token Reuse Attack (Anti-Replay)

Si un atacante intenta reutilizar un refresh token que ya fue marcado como `IsUsed = true`, el sistema detecta inmediatamente la anomalía de seguridad, **revoca en cascada todos los tokens activos del usuario** y devuelve `403 Forbidden`.

3.2. Claims Enriquecidos y RBAC

El Access Token emitido incluye información de contexto para evitar consultas repetitivas a la base de datos:

- `sub / NameIdentifier` : Identificador único del usuario (GUID).
- `email` : Correo electrónico verificado.
- `firstName / lastName` : Nombre y apellido del usuario.
- `ClaimTypes.Role & roles` : Lista de roles asignados (`Administrator`, `User`).

3.3. Middlewares de Seguridad OWASP

Encabezado de Seguridad	Valor Configurado	Ataque Mitigado
<code>X-Content-Type-Options</code>	<code>nosniff</code>	MIME type sniffing attacks
<code>X-Frame-Options</code>	<code>DENY</code>	Clickjacking & UI redressing
<code>Referrer-Policy</code>	<code>strict-origin-when-cross-origin</code>	Fuga de información confidencial en URLs
<code>Permissions-Policy</code>	<code>geolocation=(), camera=(), microphone=()</code>	Uso indebido de hardware del cliente
<code>Content-Security-Policy</code>	Hardened con soporte para Scalar Docs	Cross-Site Scripting (XSS) y code injection

3.4. Rate Limiting Anti-DDoS

Configurado a través de `RateLimiter` nativo de ASP.NET Core: particiona por dirección IP remota y restringe a **100 solicitudes por minuto**, respondiendo `429 Too Many Requests` ante ráfagas no autorizadas.

4. Persistencia, Base de Datos & Resiliencia

4.1. Entity Framework Core 10 & PostgreSQL 17

La persistencia se gestiona con `ApplicationDbContext` implementando la interfaz contractual `IApplicationDbContext`. Cada entidad tiene su configuración Fluent API aislada en `Infrastructure/Persistence/Configurations/`.

4.2. Auditoría y Soft Delete Automático

El interceptor `AuditableViewEntitySaveChangesInterceptor` automatiza la trazabilidad:

- **Inserción:** Asigna `CreatedAt =.UtcNow` y `CreatedBy = CurrentUserId`.
- **Modificación:** Asigna `LastModifiedAt =.UtcNow` y `LastModifiedBy = CurrentUserId`.
- **Eliminación (Soft Delete):** Modifica el estado a `Modified`, asignando `IsDeleted = true`, `DeletedAt =.UtcNow` y `DeletedBy = CurrentUserId`.
- **Consultas automáticas:** Todas las consultas excluyen registros eliminados gracias a `builder.HasQueryFilter(e => !e.IsDeleted)`.

4.3. Resiliencia de Conexión con Polly v8

Se configuran reintentos automáticos con retroceso exponencial ante fallos transitorios de red o reinicios de PostgreSQL:

```
npgsqlOptions.EnableRetryOnFailure(  
    maxRetryCount: 3,  
    maxRetryDelay: TimeSpan.FromSeconds(5),  
    errorCodesToAdd: null);
```

5. Guía Paso a Paso: Cómo Agregar un Nuevo Módulo / Feature

A continuación se ilustra el procedimiento para crear un módulo de **Productos** (`Product`) en menos de 10 minutos:

Paso 1: Entidad de Dominio (`BackendTemplate.Domain/Entities/Product.cs`)

```
using BackendTemplate.Domain.Common;

namespace BackendTemplate.Domain.Entities;

public class Product : AuditableEntity, ISoftDeletable
{
    public string Name { get; private set; } = string.Empty;
    public string Sku { get; private set; } = string.Empty;
    public decimal Price { get; private set; }
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
    public string? DeletedBy { get; set; }

    private Product() { }

    public Product(string name, string sku, decimal price)
    {
        Id = Guid.NewGuid();
        Name = name;
        Sku = sku;
        Price = price;
    }

    public void Delete(string? deletedBy = null)
    {
        IsDeleted = true;
        DeletedAt = DateTime.UtcNow;
        DeletedBy = deletedBy;
    }
}
```

Paso 2: Contrato y Caso de Uso (`BackendTemplate.Application`)

1. Agrega `DbSet<Product> Products { get; }` en `IApplicationDbContext.cs`.
2. Crea el Command, Validator y Handler:


```

public record CreateProductCommand(string Name, string Sku, decimal Price) : IRequest<Result<Guid>>

public class CreateProductCommandValidator : AbstractValidator<CreateProductCommand>
{
    public CreateProductCommandValidator()
    {
        RuleFor(v => v.Name).NotEmpty().MaximumLength(150);
        RuleFor(v => v.Sku).NotEmpty().MaximumLength(50);
        RuleFor(v => v.Price).GreaterThan(0);
    }
}

public class CreateProductCommandHandler : IRequestHandler<CreateProductCommand, Result<Guid>>
{
    private readonly IApplicationDbContext _context;
    public CreateProductCommandHandler(IApplicationDbContext context) => _context = context;

    public async Task<Result<Guid>> Handle(CreateProductCommand request, CancellationToken cancel
    {
        var product = new Product(request.Name, request.Sku, request.Price);
        await _context.Products.AddAsync(product, cancellationTokens);
        await _context.SaveChangesAsync(cancellationTokens);
        return Result<Guid>.Success(product.Id);
    }
}

```

Paso 3: Controlador REST (`BackendTemplate.Api/Controllers/ProductsController.cs`)

```

[Authorize]
public class ProductsController : ApiControllerBase
{
    [HttpPost]
    [Authorize(Roles = "Administrator")]
    public async Task<IActionResult> Create([FromBody] CreateProductCommand command)
    {
        return HandleResult(await Mediator.Send(command));
    }
}

```

6. Estrategia de Pruebas & Calidad de Código

La solución cuenta con **29 pruebas automatizadas** que garantizan que el código cumple con las reglas de arquitectura y negocio:

Categoría	Herramienta	Objetivo de Cobertura
Pruebas de Arquitectura	NetArchTest.Rules	Garantiza que ninguna capa viole la regla de dependencia de Clean Architecture y verifica convenciones de nombrado.
Pruebas Unitarias	xUnit + Moq + FluentAssertions	Prueba reglas de validación en FluentValidation y lógica de negocio en Handlers CQRS.
Pruebas de Integración	WebApplicationFactory<Program>	Ejecuta peticiones HTTP reales contra el pipeline completo (Routing, Middlewares, Auth, Controllers).

Ejecución de Pruebas

```
# Ejecutar toda la suite de pruebas
dotnet test

# Ejecutar con reporte de cobertura
dotnet test --collect:"XPlat Code Coverage"
```

7. Despliegue, Contenedores & Operaciones (DevOps)

7.1. Despliegue con Docker Compose

El archivo `docker-compose.yml` orquesta la infraestructura completa:

```
# Iniciar PostgreSQL 17, Redis 7, Mailpit SMTP y la API
docker compose up -d --build
```

7.2. Dockerfile Multi-Stage de Producción

- **Base:** Imagen oficial de runtime de Microsoft .NET 10.
- **Seguridad:** Ejecución bajo usuario no privilegiado `USER app` (UID 1654).
- **Optimización:** Multi-stage build que reduce la imagen final a menos de 150 MB.

7.3. Health Checks para Balanceadores y Kubernetes

- `GET /health` : Endpoint de infraestructura para liveness/readiness probes (devuelve `200 OK Healthy`).
- `GET /api/health` : Endpoint JSON estandarizado para monitoreo de aplicación.

8. Herramientas de Productividad (DX) & Referencia de API

8.1. Documentación Interactiva Scalar

- **URL:** `http://localhost:5000/scalar/v1` o `https://localhost:5001/scalar/v1`
- Soporte nativo para autenticación **Bearer JWT**.
- Generador de código cliente para múltiples lenguajes (C#, TypeScript, Python, cURL).

8.2. Cliente REST Integrado (`backend-requests.http`)

Compatible con Visual Studio y VS Code REST Client. Captura automáticamente el token JWT tras el login y lo propaga a las peticiones protegidas:

```
### Login y Captura de Token
# @name loginAdmin
POST {{host}}/api/auth/login
Content-Type: application/json

{
  "email": "admin@template.com",
  "password": "Admin123!"
}

### Consulta Autenticada con el Token Capturado
@authToken = {{loginAdmin.response.body.data.accessToken}}
GET {{host}}/api/students?pageNumber=1&pageSize=10
Authorization: Bearer {{authToken}}
```

8.3. Credenciales Semilla por Defecto

Usuario	Email	Contraseña	Rol Asignado
Administrador del Sistema	admin@template.com	Admin123!	Administrator, User